Author: Ganesh Babu G

SAS® vs R / Tidyverse / dplyr

Complete Clinical Data Programming Reference for SDTM & ADaM

SAS® — Procedure / Data Step	**R tidyverse / dplyr** — Function / Package	**admiral** — Pharmaverse Clinical R Package

1. READING & IMPORTING DATA

Operation	SAS®	R / Tidyverse / dplyr
Read SAS dataset (.sas7bdat)	`LIBNAME sdtm '...'; data dm; set sdtm.dm; run;`	`library(haven) dm <- read_sas('sdtm/dm.sas7bdat')`
Read XPT transport file	`LIBNAME xlib xport 'dm.xpt'; data dm; set xlib.dm; run;`	`dm <- haven::read_xpt('dm.xpt')`
Read CSV	`PROC IMPORT datafile='data.csv' out=mydata dbms=csv replace; run;`	`mydata <- readr::read_csv('data.csv') # or: read.csv('data.csv')`
Read Excel	`PROC IMPORT datafile='data.xlsx' out=mydata dbms=xlsx replace; run;`	`mydata <- readxl::read_xlsx('data.xlsx')`
Write XPT for submission	`LIBNAME out xport 'adsl.xpt'; data out.adsl; set adsl; run;`	`library(xportr) adsl |> xportr_write('adsl.xpt')`
Set LIBNAME / assign path	`LIBNAME sdtm '/path/to/sdtm';`	`sdtm <- '/path/to/sdtm' dm <- read_sas(file.path(sdtm,'dm.sas7bdat'))`
Convert blanks to NA	`(Blanks are missing by default in SAS)`	`admiral::convert_blanks_to_na(dm)`

haven::read_sas() preserves SAS formats and labels as R attributes. xportr applies CDISC metadata back before writing XPT.

2. DATA STEP & BASIC MANIPULATION (mutate / select / filter)

Operation	SAS® DATA Step	R dplyr / tidyverse
Create / derive new variable	`data adsl; set adsl; age_grp = 'YOUNG'; if age >= 65 then age_grp = 'ELDERLY'; run;`	`adsl <- adsl |> mutate(age_grp = if_else(age >= 65, 'ELDERLY', 'YOUNG'))`
Multiple derivations	`data work; set dm; bmi = weight/(height/100)**2; saffl = 'Y'; run;`	`work <- dm |> mutate( bmi = weight/(height/100)^2, saffl = 'Y' )`
Keep specific columns	`data out; set dm (keep = usubjid age sex); run;`	`out <- dm |> select(usubjid, age, sex)`
Drop specific columns	`data out; set dm (drop = dmdtc); run;`	`out <- dm |> select(-dmdtc)`
Filter / subset rows (WHERE)	`data safpop; set adsl; where saffl = 'Y'; run;`	`safpop <- adsl |> filter(saffl == 'Y')`
Multiple filter conditions	`where saffl='Y' and age >= 18 and sex in ('M','F');`	`filter(saffl=='Y', age>=18, sex %in% c('M','F'))`
Rename variable	`data out; set dm (rename=(arm=trt01p)); run;`	`out <- dm |> rename(trt01p = arm)`

Author: Ganesh Babu G

Operation	SAS® DATA Step	R dplyr / tidyverse
Recode / map values	`if sex='M' then sex='MALE'; if sex='F' then sex='FEMALE';`	`mutate(sex = case_when( sex=='M'~'MALE', sex=='F'~'FEMALE', TRUE~sex))`
Conditional assignment (IFN/IFC)	`saffl = ifn(safety_pop=1,'Y','N');`	`mutate(saffl = if_else(safety_pop==1,'Y','N'))`
Retain / carry-forward	`retain lastdt; if not missing(adt) then lastdt=adt;`	`mutate(lastdt = if_else(!is.na(adt), adt, lag(adt))) # or: tidyr::fill(lastdt, .direction='down')`
Array / vector recoding	`array trt{3} trt1-trt3; do i=1 to 3; if trt{i}=. then trt{i}=0; end;`	`mutate(across(starts_with('trt'), ~replace_na(.x, 0)))`

3. SORTING (PROC SORT arrange)

Operation	SAS®	R dplyr
Sort ascending	`proc sort data=adsl; by usubjid; run;`	`adsl <- adsl |> arrange(usubjid)`
Sort descending	`proc sort data=adae; by usubjid descending aestdtc; run;`	`adae <- adae |> arrange(usubjid, desc(aestdtc))`
Sort by multiple keys	`by usubjid visitnum paramcd;`	`arrange(usubjid, visitnum, paramcd)`
Remove duplicates after sort	`proc sort data=ae noduprecs; by usubjid aeseq; run;`	`ae |> arrange(usubjid,aeseq) |> distinct(usubjid, aeseq, .keep_all=TRUE)`

4. BY-GROUP PROCESSING (BY Statement group_by)

Operation	SAS®	R dplyr
First / last obs in group	`proc sort data=ex; by usubjid exstdtc; run; data ex_first; set ex; by usubjid; if first.usubjid; run;`	`ex_first <- ex |> arrange(usubjid, exstdtc) |> group_by(usubjid) |> slice_head(n=1) |> ungroup()`
Count per group	`proc freq data=adae; tables usubjid / out=cnt; run;`	`cnt <- adae |> count(usubjid, name='n_ae')`
Group summary statistics	`proc means data=advs; by usubjid paramcd; var aval; output out=summ mean=mean_aval; run;`	`summ <- advs |> group_by(usubjid, paramcd) |> summarise(mean_aval=mean(aval,na.rm=T), .groups='drop')`
Row number within group	`data ae; set ae; by usubjid; if first.usubjid then seq=0; seq+1; run;`	`ae <- ae |> group_by(usubjid) |> mutate(aeseq=row_number()) |> ungroup()`
Cumulative max within group	`data out; set ds; by usubjid; retain max_aval; if first.usubjid then max_aval=.; max_aval=max(max_aval,aval); run;`	`out <- ds |> group_by(usubjid) |> mutate(max_aval=cummax(aval)) |> ungroup()`
Last non-missing value per group	`(Use RETAIN + BY group logic)`	`group_by(usubjid) |> mutate(last_val=last(na.omit(aval))) |> ungroup()`

Operation	SAS®	R dplyr
Lag within group (previous visit)	`data out; set advs; by usubjid paramcd; lagaval = lag(aval); if first.usubjid or first.paramcd then lagaval=.; run;`	`advs |> group_by(usubjid, paramcd) |> mutate(lagaval = lag(aval)) |> ungroup()`

5. MERGING & JOINING DATASETS (PROC SQL / DATA MERGE joins)

Operation	SAS®	R dplyr / admiral
Left join (1:1 or 1:M)	`proc sql; create table out as select a.*, b.trt01p from adsl a left join dm b on a.usubjid=b.usubjid; quit;`	`out <- adsl |> left_join(dm |> select(usubjid,trt01p), by='usubjid')`
DATA STEP merge (sorted)	`data merged; merge ae(in=a) adsl(in=b keep=usubjid saffl); by usubjid; if a; run;`	`merged <- ae |> left_join(adsl |> select(usubjid,saffl), by='usubjid')`
Inner join	`proc sql; create table out as select a.* from ds a inner join adsl b on a.usubjid=b.usubjid; quit;`	`out <- ds |> inner_join(adsl, by='usubjid')`
Merge SDTM + SUPP domain	`proc sort data=suppds; by usubjid; data ds_all; merge ds suppds; by usubjid; run;`	`# admiral approach: ds_all <- ds |> derive_vars_suppqual(suppds)`
Merge with admiral (add ADSL vars)	`proc sql; create table advs as select a.*, b.trtsdt, b.trtedt from vs a left join adsl b on a.usubjid=b.usubjid; quit;`	`advs <- vs |> derive_vars_merged( dataset_add = adsl, new_vars = exprs(TRTSDT,TRTEDT), by_vars = exprs(STUDYID,USUBJID) )`
Anti join (keep non-matches)	`data not_matched; merge a(in=ina) b(in=inb); by key; if ina and not inb; run;`	`not_matched <- a |> anti_join(b, by='key')`
Stack datasets (SET)	`data allae; set ae1 ae2 ae3; run;`	`allae <- bind_rows(ae1, ae2, ae3)`
Merge by multiple keys	`merge a b; by usubjid visitnum paramcd;`	`left_join(a, b, by=c('usubjid', 'visitnum','paramcd'))`

6. DATE & TIME HANDLING (lubridate + admiral)

Operation	SAS®	R lubridate / admiral
Convert DTC char SAS date	`adt = input(substr(aestdtc,1,10), yymmdd10.); format adt date9.;`	`adt = as.Date(substr(aestdtc,1,10), '%Y-%m-%d') # lubridate: ymd(substr(aestdtc,1,10))`
Convert DTC ADaM date (admira l)	(manual input/put logic required)	`derive_vars_dt( dtc = AESTDTC, new_vars_prefix = 'AST' ) # creates ASTDT + ASTDTF flag`
Convert DTC datetime (ADTM)	`astdtm = input(aestdtc, e8601dz22.); format astdtm datetime.;`	`derive_vars_dtm( dtc = AESTDTC, new_vars_prefix = 'AST', highest_imputation = 'h', time_imputation = 'first' ) # creates ASTDTM + ASTDTF + ASTTMF`
Date imputation (first/last)	`if length(aestdtc)=7 then astdt = input(cats(aestdtc,'-01'), yymmdd10.);`	`derive_vars_dt( dtc = AESTDTC, new_vars_prefix = 'AST', highest_imputation = 'M', date_imputation = 'first' )`

Operation	SAS®	R lubridate / admiral
Derive ADY (analysis day)	`ady = adt - trtsdt + (adt >= trtsdt);`	`derive_vars_dy( reference_date = TRTSDT, source_vars = exprs(ADT) )`
Compute duration in days	`dur = intck('day', start, end) + 1;`	`dur = as.integer(end - start) + 1 # lubridate: time_length(interval(start,end),'days')+1`
Compute age in years	`age = int((rfstdtc - dthdtc)/365.25);`	`age = floor(time_length( interval(dthdtc, rfstdtc), 'years')) # admiral: compute_age_years()`
Extract year/month/day parts	`yr = year(rfstdtc); mo = month(rfstdtc); day = day(rfstdtc);`	`yr = year(rfstdtc) mo = month(rfstdtc) day = day(rfstdtc) # lubridate`
Impute to not exceed death date	`(manual min logic required)`	`derive_vars_dtm( dtc = AEENDTC, new_vars_prefix = 'AEN', highest_imputation = 'Y', date_imputation = 'last', max_dates = exprs(DTHDT, DCUTDT) )`

7. SDTM-SPECIFIC DERIVATIONS

SDTM Task	SAS®	R / admiral
Assign STUDYID, DOMAIN	`data ae; set raw_ae; studyid = 'STUDY01'; domain = 'AE'; run;`	`ae <- raw_ae |> mutate(studyid='STUDY01', domain='AE')`
Derive USUBJID	`usubjid = catx('-',studyid,siteid,subjid);`	`mutate(usubjid=paste(studyid,siteid,subjid,sep='-'))`
Assign sequence number (--SEQ)	`data ae; set ae; by usubjid; if first.usubjid then aeseq=0; aeseq+1; run;`	`ae <- ae |> group_by(usubjid) |> mutate(aeseq = row_number()) |> ungroup()`
Merge SUPP domain to parent	`proc sort data=suppae; by usubjid idvarval; data ae_full; merge ae suppae; by usubjid; run;`	`ae_full <- ae |> derive_vars_suppqual(suppae)`
Map raw term to controlled term	`if raw_sex='Male' then sex='M'; if raw_sex='Female' then sex='F';`	`mutate(sex = case_when( raw_sex=='Male' ~ 'M', raw_sex=='Female'~ 'F', TRUE ~ NA_character_))`
Apply controlled terminology (CT)	`(format or if/else mapping)`	`metatools::create_var_from_codelist( metacore_obj, dataset, var_name )`
Pivot raw multi-row to SDTM BDS	`proc transpose data=vs out=vs_long(rename=(_name_=vstestcd col1=vsorres)); by usubjid visitnum; run;`	`vs_long <- vs_wide |> pivot_longer( cols = c(height, weight, bmi), names_to = 'vstestcd', values_to = 'vsorres' )`
Sort SDTM per define order	`proc sort data=ae; by studyid usubjid aeseq; run;`	`ae <- ae |> arrange(studyid, usubjid, aeseq)`
Convert numeric date to ISO DTC	`aestdtc = put(aestdt, yymmdd10.);`	`mutate(aestdtc = format(aestdt, '%Y-%m-%d'))`

8. ADaM DERIVATIONS (admiral pharmaverse package)

The {admiral} package provides CDISC-compliant derivation functions for all major ADaM datasets.

Author: Ganesh Babu G

8a. ADSL — Subject Level Analysis Dataset

ADaM Variable	SAS® Approach	R / admiral Function
TRT01P / TRT01A (treatment assignment)	`trt01p = arm; trt01a = actarm;`	`adsl <- dm |> mutate(TRT01P=ARM, TRT01A=ACTARM)`
TRTSDT / TRTEDT (treatment dates)	`(merge EX domain + min/max logic)`	`derive_vars_merged( dataset_add = ex_ext, new_vars = exprs(TRTSDTM=EXSTDTM), order = exprs(EXSTDTM, EXSEQ), mode = 'first', by_vars = exprs(STUDYID, USUBJID) )`
RANDDT (randomisation date)	`proc sql; select min(dsstdtc) from ds where dsdecod='RANDOMIZED';`	`derive_vars_dt( new_vars_prefix='DS', dtc=DSSTDTC ) |> derive_vars_merged( dataset_add=ds_ext, filter_add= DSDECOD=='RANDOMIZED', new_vars=exprs(RANDDT=DSSTDT) )`
EOSDT (end of study date)	`(min of disposition date by subject)`	`derive_vars_dt( new_vars_prefix='EOS', dtc=DSSTDTC ) # then derive_vars_merged()`
DTHDT (death date)	`dthdt=input(dthdtc, yymmdd10.);`	`derive_vars_dt( new_vars_prefix='DTH', dtc=DTHDTC, date_imputation='first' )`
AGEGR1 (age group)	`if age<65 then agegr1='<65'; else agegr1='>=65';`	`mutate(agegr1=case_when( age<65 ~ '<65', age>=65 ~ '>=65' ))`
SAFFL / ITTFL (population flags)	`saffl = 'Y'; if not missing(randdt) then ittfl='Y';`	`mutate( SAFFL='Y', ITTFL=if_else(!is.na(RANDDT),'Y','N') )`
DCSREAS (discontinuation reason)	`(manual lookup from DS domain)`	`derive_vars_extreme_event( source = event_source(ds, filter=DSCAT=='DISPOSITION EVENT', set_values_to=exprs( DCSREAS=DSDECOD )), by_vars=exprs(STUDYID,USUBJID) )`

8b. BDS Datasets — ADVS, ADLB, ADAE (findings)

ADaM Derivation	SAS® Approach	R / admiral Function
Map PARAMCD from TESTCD lookup	`proc sql; create table advs as select a.*, b.paramcd, b.param from vs a left join param_lkp b on a.vstestcd=b.vstestcd; quit;`	`derive_vars_merged_lookup( dataset_add = param_lookup, new_vars = exprs(PARAMCD), by_vars = exprs(VSTESTCD) )`
Derive ADT / ADTM from DTC	`adt = input(substr(vsdtc,1,10), yymmdd10.); format adt date9.;`	`derive_vars_dt( dtc=VSDTC, new_vars_prefix='A' ) # creates ADT + ADTF`
Derive ADY (relative day)	`if adt >= trtsdt then ady = adt-trtsdt+1; else ady = adt-trtsdt;`	`derive_vars_dy( reference_date=TRTSDT, source_vars=exprs(ADT) )`

ADaM Derivation	SAS® Approach	R / admiral Function
Derive ABLFL (baseline flag)	`data advs; set advs; by usubjid paramcd; if last.paramcd and adt<=trtsdt then ablfl='Y'; run;`	`restrict_derivation( derivation =derive_var_extreme_flag, args=params( by_vars=exprs(USUBJID,PARAMCD), order=exprs(ADT,VISITNUM), new_var=ABLFL, mode='last' ), filter=(!is.na(AVAL) & ADT<=TRTSDT) )`
Derive BASE (baseline value)	`data advs; set advs; by usubjid paramcd; retain base; if ablfl='Y' then base=aval; run;`	`derive_var_base( by_vars=exprs( STUDYID,USUBJID,PARAMCD), source_var=AVAL, new_var=BASE )`
Derive CHG (change from baseline)	`chg = aval - base;`	`derive_var_chg()`
Derive PCHG (% change)	`pchg = (aval-base)/base*100;`	`derive_var_pchg()`
Add derived parameter (MAP)	`(manual calculation row-level)`	`derive_param_computed( parameters=c('DIABP','SYSBP'), set_values_to=exprs( PARAMCD='MAP', AVAL=(2*AVAL.DIABP+AVAL.SYSBP)/3 ) )`
Derive ANRIND / BNRIND (normal range)	`if aval < anrlo then anrind='LOW'; else if aval > anrhi then anrind='HIGH'; else anrind='NORMAL';`	`derive_var_anrind( signif_dig=3 ) # then derive_var_base for BNRIND`
Treatment-emergent AE flag (TRTEMFL)	`if astdt >= trtsdt and (aeendt <= trtedt or missing(aeendt)) then trtemfl='Y';`	`derive_var_trtemfl( new_var=TRTEMFL, trt_start_date=TRTSDT, trt_end_date=TRTEDT, end_window=NULL )`

8c. ADSL / BDS — Higher-Order admiral Functions

Function	Purpose	Clinical Use Case
`restrict_derivation()`	Apply a derivation only to a filtered subset of the dataset	Derive ABLFL only for pre-treatment records
`call_derivation()`	Call the same derivation multiple times with varying arguments	Derive multiple period treatment dates
`slice_derivation()`	Apply different derivations to different subsets	Different baseline rules by PARAMCD
`derive_vars_extreme_event()`	Derive variables from the first/last event across multiple sources	Cause of death from AE/DS/DD domains
`derive_var_extreme_flag()`	Flag the first or last record per group	ABLFL, DTYPE=LOCF flags
`derive_vars_merged()`	Merge variables from another dataset into the current one	Add ADSL variables to BDS datasets
`derive_vars_merged_lookup()`	Merge using a lookup/parameter mapping table	Map VSTESTCD PARAMCD/PARAM
`derive_param_exist_flag()`	Add a flag indicating if a record exists for a parameter	Derive population-level response flags
`derive_basetype_records()`	Add BASETYPE records for multiple baseline definitions	On-treatment / last-on-treatment baselines

■ 9. AGGREGATION & SUMMARY STATISTICS (PROC MEANS/FREQ/TABULATE)

Operation	SAS®	R dplyr / tidyverse
Basic descriptive stats	`proc means data=advs; by trt01pn paramcd; var aval; output out=summ n=n mean=mean std=std min=min max=max; run;`	`summ <- advs |> group_by(trt01pn, paramcd) |> summarise( n=sum(!is.na(aval)), mean=mean(aval,na.rm=T), sd=sd(aval,na.rm=T), min=min(aval,na.rm=T), max=max(aval,na.rm=T), .groups='drop' )`
Frequency count	`proc freq data=adae; tables trtemfl; run;`	`adae |> count(trtemfl)`
Cross-tabulation	`proc freq data=adae; tables sex * agegr1 / nocol nopercent; run;`	`adae |> count(sex, agegr1) |> pivot_wider(names_from=agegr1, values_from=n, values_fill=0)`
Median and quantiles	`proc means data=advs median q1 q3;`	`summarise( median=median(aval,na.rm=T), q1=quantile(aval,.25,na.rm=T), q3=quantile(aval,.75,na.rm=T) )`
Summary across multiple vars	`proc means data=adsl; var age weight height; run;`	`adsl |> summarise(across( c(age,weight,height), list(mean=~mean(.x,na.rm=T), sd=~sd(.x,na.rm=T)) ))`
N subjects with event	`proc sql; select count(distinct usubjid) from adae where trtemfl='Y'; quit;`	`adae |> filter(trtemfl=='Y') |> distinct(usubjid) |> nrow()`

■ 10. TRANSPOSING & RESHAPING (PROC TRANSPOSE pivot_*)

Operation	SAS®	R tidyr
Wide Long (vertical/unpivot)	`proc transpose data=adsl out=long name=var label=label; by usubjid; var age weight height; run;`	`long <- adsl |> pivot_longer( cols=c(age,weight,height), names_to='var', values_to='value' )`
Long Wide (horizontal/pivot)	`proc transpose data=advs out=wide(drop=_name_) prefix=visit; by usubjid paramcd; id visitnum; var aval; run;`	`wide <- advs |> pivot_wider( id_cols=c(usubjid,paramcd), names_from=visitnum, names_prefix='visit', values_from=aval )`
SUPP domain to wide columns	`proc transpose data=suppae out=supp_wide(drop=_name_); by usubjid idvarval; id qnam; var qval; run;`	`supp_wide <- suppae |> pivot_wider( id_cols=c(usubjid,idvarval), names_from=qnam, values_from=qval )`
Separate single column	`arm_grp = scan(arm,1,' '); arm_sub = scan(arm,2,' ');`	`tidyr::separate(arm, c('arm_grp','arm_sub'), sep=' ', extra='merge')`
Unite/concatenate columns	`usubjid = catx('-',studyid,siteid,subjid);`	`tidyr::unite(usubjid, studyid,siteid,subjid, sep='-')`
Fill missing (LOCF)	`data locf; set advs; by usubjid paramcd; retain laval; if not missing(aval) then laval=aval; if missing(aval) then aval=laval; run;`	`advs |> group_by(usubjid, paramcd) |> mutate(aval=tidyr::fill(aval, .direction='down')) |> ungroup()`

■ 11. STRING & CHARACTER FUNCTIONS (stringr)

Operation	SAS®	R stringr / base
Substring extraction	`substr(aestdtc,1,4) /* year */ substrn(aestdtc,1,4)`	`stringr::str_sub(aestdtc,1,4) # base: substr(aestdtc,1,4)`
Concatenate strings	`usubjid = cats(studyid,'-',siteid); /* or catx('-',studyid,siteid) */`	`paste0(studyid,'-',siteid) paste(studyid,siteid,sep='-')`
Convert to UPPER / lower	`upcase(sex) lowcase(sex)`	`toupper(sex) tolower(sex) stringi::stri_trans_toupper()`
Strip leading/trailing spaces	`strip(var) trim(var)`	`stringr::str_trim(var) trimws(var)`
Detect pattern	`if index(aeterm,'HEADACHE')>0;`	`stringr::str_detect(aeterm,'HEADACHE')`
Replace pattern	`tranwrd(aeterm,'HEADACHE','MIGRAINE')`	`stringr::str_replace(aeterm,'HEADACHE','MIGRAINE')`
Length of string	`length = lengthn(var);`	`nchar(var) stringr::str_length(var)`
Left-pad with zeros	`putn = put(seq,z3.); /* 001, 002 */`	`stringr::str_pad(seq, 3, pad='0') formatC(seq, width=3, flag='0')`
Extract numbers from string	`num = input(compress(str,'','kd'),best.);`	`as.numeric(stringr::str_extract(str,'[0-9.]+'))`
Split string into parts	`part1 = scan(str,1,'-');`	`stringr::str_split(str,'-')[[1]][1]`

12. MACROS, LOOPS & ITERATION

Pattern	SAS® Macro Language	R Functional Approach
Macro variable / global variable	`%let studyid = STUDY01; %put &studyid;`	`studyid <- 'STUDY01' cat(studyid)`
Loop over list of datasets	`%macro loop; %do i=1 %to 3; proc print data=ds&i; run; %end; %mend; %loop;`	`purrr::walk( list(ds1,ds2,ds3), ~print(.x) ) # or: lapply(list(ds1,ds2,ds3), print)`
Apply function to multiple vars	`%macro derive(var); new_&var; = &var;*2; %mend; %derive(weight); %derive(height);`	`mutate(across( c(weight,height), ~.x*2, .names='new_{.col}' ))`
Read multiple domains from loop	`%macro read(dom); data &dom; set sdtm.&dom; run; %mend; %read(ae); %read(lb);`	`domains <- c('ae','lb','vs') purrr::walk(domains, function(d) { assign(d, haven::read_sas(file.path(sdtm, paste0(d,'.sas7bdat'))), envir=.GlobalEnv) })`
Conditional macro logic	`%if &safety;=Y %then %do; /* run safety analysis */ %end;`	`if (safety == 'Y') { # run safety analysis }`
Create reusable function	`%macro derive_flag(ds, var, flag); data &ds; set &ds; if not missing(&var;) then &flag;='Y'; run; %mend;`	`derive_flag <- function(df, var, flag_nm) { df |> mutate( {{flag_nm}} := if_else(!is.na({{var}}),'Y','N') ) }`

13. VALIDATION, QC & METADATA

Task	SAS®	R / pharmaverse
Compare two datasets	`proc compare base=qc.adsl compare=work.adsl; run;`	`diffdf::diffdf(qc_adsl, adsl) # or: arsenal::comparedf()`
Check row counts	`proc sql; select count(*) from adsl; quit;`	`nrow(adsl) adsl |> count()`
Check unique keys	`proc sort data=adsl noduprecs; by usubjid; run;`	`adsl |> distinct(usubjid) |> nrow() # or: nrow(adsl)==n_distinct(adsl$usubjid)`
Apply variable labels	`label usubjid='Unique Subject Identifier' age='Age in Years';`	`labelled::var_label(adsl) <- list( usubjid='Unique Subject Identifier', age='Age in Years' ) # or: metacore + metatools`
Apply dataset label / format	`data adsl(label='Subject-Level Analysis'); set adsl; run;`	`attr(adsl,'label') <- 'Subject-Level Analysis Dataset' # xportr::xportr_label(adsl, metacore)`
Validate SDTM/ADaM (Pinnacle21)	`(Pinnacle21 external tool, SAS XPT output)`	`xportr::xportr_write(adsl,'adsl.xpt') # then submit to Pinnacle21 Community`
Apply metadata from spec sheet	`(manual label/format statements)`	`metacore::spec_to_metacore('spec.xlsx') metatools::set_variable_labels(adsl, metacore_obj)`
Unit testing / peer QC	`(log review, proc compare)`	`testthat::test_that('SAFFL is Y/N', { expect_true(all(adsl$SAFFL %in% c('Y','N'))) })`
Write XPT with metadata	`LIBNAME out xport 'adsl.xpt'; data out.adsl; set adsl; run;`	`adsl |> xportr_label(metacore_obj) |> xportr_type(metacore_obj) |> xportr_write('adsl.xpt')`

14. CLINICAL R PACKAGE ECOSYSTEM (pharmaverse)

Package	Purpose / SAS Equivalent	Key Functions
`admiral`	ADaM derivation library — replaces most custom SAS DATA step macro code	`derive_vars_dt/dtm(), derive_var_base(), derive_var_chg(), derive_vars_merged(), derive_var_extreme_flag()`
`haven`	Read/write SAS .sas7bdat & .xpt files — replaces LIBNAME + PROC IMPORT	`read_sas(), write_xpt(), read_xpt()`
`metacore`	Store and use dataset metadata/specs — replaces manual label/format coding	`spec_to_metacore(), select_dataset()`
`metatools`	Build datasets using metacore metadata — replaces label statements + format catalogs	`set_variable_labels(), create_var_from_codelist()`
`xportr`	Apply metadata & write CDISC-compliant XPT — replaces LIBNAME xport	`xportr_label(), xportr_type(), xportr_write()`
`pharmaversesdtm`	CDISC Pilot SDTM example datasets for testing	`dm, ae, vs, lb, ex, ds, ...`
`diffdf`	Dataset comparison — replaces PROC COMPARE	`diffdf(base, compare)`

Package	Purpose / SAS Equivalent	Key Functions
`lubridate`	Date/time manipulation — replaces SAS date functions (INTNX, INTCK, INPUT)	`ymd(), interval(), time_length(), year(), month(), day()`
`stringr`	String manipulation — replaces SUBSTR, CATX, COMPRESS, TRANWRD	`str_sub(), str_detect(), str_replace(), str_trim()`
`tidyr`	Reshape data — replaces PROC TRANSPOSE	`pivot_longer(), pivot_wider(), fill(), separate(), unite()`
`readr / readxl`	Read CSV/Excel specs — replaces PROC IMPORT	`read_csv(), read_xlsx()`
`purrr`	Functional iteration — replaces SAS %DO loops and macro-driven dataset processing	`map(), walk(), map_dfr()`
`tplyr`	TFL table generation — replaces PROC TABULATE/REPORT	`tplyr_table(), add_layer()`
`rtables / rlistings`	Clinical tables and listings — replaces PROC REPORT	`build_table(), as_listing()`

15. QUICK REFERENCE — SAS R ONE-LINERS

SAS Operation	SAS®	R Equivalent
KEEP statement	`keep usubjid age sex;`	`select(usubjid, age, sex)`
DROP statement	`drop dmdtc dthdtc;`	`select(-dmdtc, -dthdtc)`
WHERE clause	`where saffl='Y';`	`filter(saffl=='Y')`
IF/THEN assignment	`if age>=65 then agegr1='ELD';`	`mutate(agegr1=if_else(age>=65,'ELD','ADULT'))`
PROC SORT	`proc sort; by usubjid visitnum;`	`arrange(usubjid, visitnum)`
BY + FIRST./LAST.	`first.usubjid / last.usubjid`	`group_by(usubjid) |> slice_head(1) / slice_tail(1)`
RETAIN	`retain prev_aval;`	`mutate(prev_aval=lag(aval))`
DATA STEP merge	`merge a b; by usubjid;`	`left_join(a, b, by='usubjid')`
SET (stack)	`set ae1 ae2;`	`bind_rows(ae1, ae2)`
PROC MEANS n/mean/std	`proc means n mean std;`	`summarise(n=n(), mean=mean(x), sd=sd(x))`
PROC FREQ tables	`proc freq; tables sex;`	`count(sex)`
PROC TRANSPOSE	`proc transpose; by id; var vals;`	`pivot_longer() / pivot_wider()`
CATS() concatenate	`cats(a,'-',b)`	`paste0(a,'-',b)`
SCAN() word	`scan(str,1,' ')`	`str_split(str,' ')[[1]][1]`
UPCASE()	`upcase(sex)`	`toupper(sex)`
STRIP()	`strip(var)`	`trimws(var)`
PUT() / INPUT()	`put(dt,date9.) / input(dtc,...)`	`format(dt,'%Y%m%d') / as.Date(dtc,'%Y-%m-%d')`
N (row count in group)	`(BY group counter)`	`group_by(...) |> mutate(n=n())`
ROW NUMBER in group	`counter+1 with RETAIN`	`group_by() |> mutate(seq=row_number())`
LAG()	`lag(aval) — careful: BY-group`	`group_by() |> mutate(prev=lag(aval))`
%LET macro var	`%let var=value;`	`var <- 'value'`

Author: Ganesh Babu G

SAS Operation	SAS®	R Equivalent
%IF/%THEN macro	`%if &cond;=Y %then %do; ... %end;`	`if (cond == 'Y') { ... }`